FIRLT : Fault Injection in Rust with Lazart Tool

Timothe Baleras
Ensimag,UGA,Verimag
timothe.baleras@grenoble-inp.org

Supervised by: Laurent Mounier,Etienne Boespflug

I understand what plagiarism entails and I declare that this report is my own, original work.
Baleras timothe, 13-08-2026 and signature :

1 Abstract

Remade Abstract

2 Introduction

introduction Remade : introduction of fault injection + Rust and Contribution

2.1 Fault injection

In computer science, many types of faults can cause the system to behave unexpectedly. Faults may occur during development or execution of the program and can originate from the software, hardware or both. Faults come in a wide variety, they can be syntax errors that cause the script to fail to compile, generated warnings, or crashed with an error during compilation. Faults can be caused by human errors, misconfiguration that were not anticipated, or malicious individuals exploiting security vulnerabilities. One important type of fault is **fault injection**. Fault injection has been around for a long time, and we found traces of paper dating from 1970s but there were never published, the oldest one published is in 1995 [1]. This fault can be caused by various techniques, including hardware such as clock glitch [2, 3], voltage glitch [4, 5, 6], laser [7, 8], or electromagnetic fault [9, 10], as well as software such as Rowhammer [11]. It can occur accidentally, for example due to cosmic rays, or deliberately by a mischievous individual, in the latter case, we talk about a fault injection attack. To deal with fault injection, developers use several countermeasures [12, 13, 14] or tools [15, 16, 1, 17] to protect their programs. One such tool is called Lazart[18]. It's a multi-fault injection tool, that mutate the code at the LLVM, in order to simulate the effects of fault injection attacks.

2.2 Rust

Rust language is a recent language and defining to be a succesor of C and C++ ; while C/C++ have a lot of memory access bug such as buffer overflow or Data races issue.

Rust have more stronger protection over memory access and concurrency over variable, variable are track with an element call the borrow checker that track the lifespan and ownership as well there acces. While the borrow checker protect bad acces of data on the software program.

The same cannot be said with the hardware failure. that may cause some protection to be counter.

add exemple ?

2.3 Contribution

The purpose of this report is to show how Rust differ with C and C++ regarding offensive fault injection. To answer that there three different part :

- **Extand Lazart to support Rust**
- **Porting the FISSC benchmark to Rust**
- **Analyzing idiomatic Rust**

explain more ?

3 Related work

In this section we talk about related work and a description of lazart tool.

3.1 Fault injection

Fault injection is an old, well-studied attack models in security and cryptology that can be created at hardware or software level[13]. While multiple methods exist multiple methods to simulate fault injection, one approach involves hardware-based techniques, such as [14, 16]. Another method to simulate fault injection is using software tools [18, 19]. One sush software tool is called Lazart [18]. A countermeasure benchmark, developed using the Lazart tool by several researchers, is FISSC [12].

While a few studies have been conducted on countermeasures in Rust [20, 21] ,there is little research

that has been conducted on comparing the results between C and Rust for countermeasures.

Furthermore, little research has been done on the effectiveness of the protecting of the countermesure from compiler optimisations. We addressed this gap by highlighting the work of Sébastien Michelland [22].

3.2 Lazart tools

Lazart [18] is a high level fault injection tool. Has been develop by Verimag since 2015. It allows to identifying vulnerable paths in code and can be used to evaluate the accuracy of countermeasures against a given fault model and a security property. Unlike other similar tools, Lazart is one of the few that are capable of performing multi-fault attacks. Lazart is able to analyse C or C++ code by compiling it using the Clang compiler. They use one of the features of Clang to generate LLVM bitecode (which is Intermediate code).
LLVM is a middle component of the compiler and is designed for compile-time, link-time and run-time optimisation. The LLVM framework is run by Clang after the frontend pipeline is completed.

Lazart is composed of five step :

1. Preprocessing and compilation of the input code.
2. Mutation of the code with Wolverine.
3. Dynamic-Symbolic Execution with KLEE [23, 24].
4. Trace parsing from KLEE's outputs.
5. Analysis.

Lazart can support 4 fault models :

— Test inversion (modification of the evaluation of a branch condition) ;
— data mutation (modification of the loaded values) ;
— instruction skip ;
— Call another function (function switch) ;

under new section

3.3 Rust Language

Rust is a general-purpose programming language which emphasizes performance, type safety, concurrency, and memory safety. Rust has is first version released in 2006. One of features of rust is the way hwo it handle the memory safety and concurrency of a program. Unlike other language who use a garbage colector, Rust use a "barrow checker" that track the life spand of a variable as there owner. And deal with data races.

Why using Rust ?

We give several reason of using rust :

— Rust use the same LLVM as clang does in C/C++. So it make Rust compatible with lazart.
— Rust is a recent,still updated programming language.
— Rust is used for embeded system.

The usage of no_std

One point that need to be explain on this reporte is the flag no_std. Using no_std attribute stops the compiler from linking std into the crate, and only links core. It limit :

— Heap allocation.
— File I/O, threading, and networking.
— Many common traits and types (e.g., std : :io : :Error).

below new section

4 RUST ported in lazart

Some modification have been made in order to support the Rust language.

Limitation of Lazart for Rust

Although Lazart works on LLVM and rustc is capable of producing LLVM,Some limitation where encountered, when trying to analyze Rust programmers with lazart.

LLVM version. The LLVM version of Lazart was up dated from LLVM 9 to LLVM 14, as the Rust toolchain matching LLVM 9 si too old for cargo to fetch crates from registry.

Test inversion extension. rustc makes an extensive use of LLVM's `select` and `switch` instructions compared to C compilers (here `clang`). Rust is capable to generate `switch` and `select` from simple branching (exemple : condition branch), even if the `switch` instructions is often use for pattern matching. The test inversion model has been extended to support `switch` and `select` LLVM instructions in order to be able to fault some control flow constructs (to match C example in FISSC) or some language constructs (e.g. pattern matching).

The Change Pointer fault model extansion

A new form of attack as been added. Since Rust borrow checker guarentees safety at compiled time. Nothing guarentees those safety to protected at runtime are they are purely static. Since developpers rely on the borrow checker for there program and to enable optimisation. One way to break the borrow checker guarentees is to consider a fault model of *change pointer* making possible to redirect a mutable reference to another. This fault is made possible with the help of the `unsafe` keyword. At lower representation levels, the fault can map a skip on an address computation, or a fault on a register or the stack.

Change Pointer fault model The Change Pointer (CP) fault model implemented in Lazart for Rust and C, redirects a reference so that it point to another object. Fault yielding a dangling or wild pointer are out of scope due to them raising a trap and become easly detectable. A redirection between two legitimate references is silent and is exactly what defeats a borrow checker invariant.

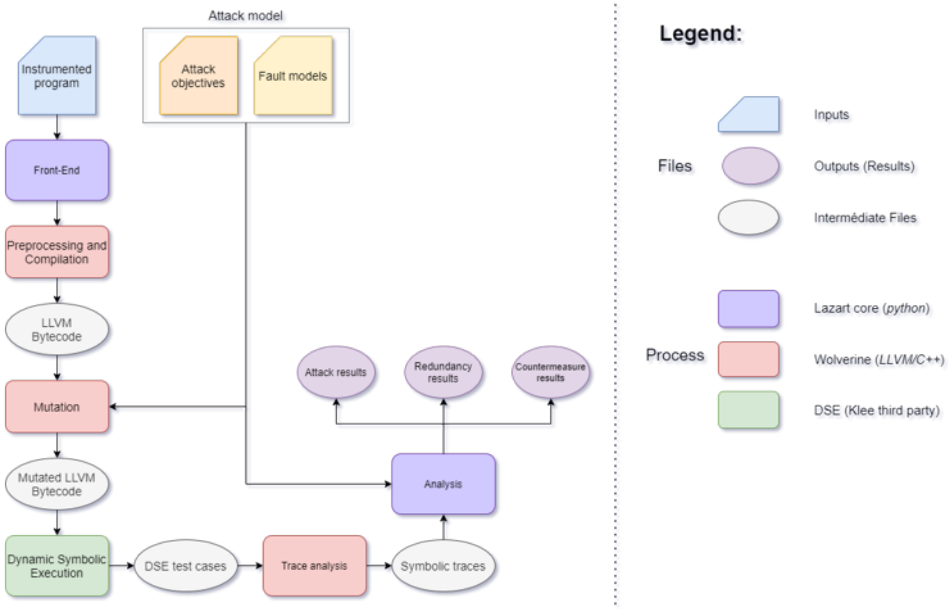

FIGURE 1 – Lazart tool arkitecture

5 Portation FISSC for lazart

REDO this section introduction

We contributed to Lazart in order by implementing Rust language to the system and to find a way to evaluate and compare the results produced by Lazart.

We evaluated two different pieces of code to see the differences in attack detected by Lazart. The first code evaluteded is `memcmps`, a secure version of `memcmp`. The second code is `verify_pin` , a program used to validate a PIN code on PIN pad. We used control-flow graphs to understand the difference between C and rust compiled code and to understand what happened during compilation and mutation of the code.

5.1 Discovery

discovery introduction + n'hésite pas à reprendre des bouts de texte rédigés dans le papier FDTC si besoin

5.2 Control flow graph

Reminder : A Control Flow Graph is a representation, using graph notation, of all paths that might be traversed through a function during its execution, or control flow. We generated a control flow graph from the generated LLVM code, which is stored in ".ll" file.

With Lazart, it is also possible to generate a control flow graph from the mutated code produced from Wolverine. We used the control-flow graphs to identify and compare the instructions used for each language (C and Rust) in order to understand their differences.

See the example in the appendix A in Figure **??**.

5.3 memcmps

```c
#include "memcmps.h"

int memcmps(char* a, char* b, int len)
{
    int result = FALSE_VAL;

    if (!memcmp(a, b, len)) {
        result ^= FALSE_VAL ^ MASK; // result = MASK
        if (!memcmp(a, b, len)) {
            result ^= MASK2; // result = MASK ^ MASK2
            if (!memcmp(a, b, len)) {
                result ^= TRUE_VAL ^ MASK; // result =
                MASK2 ^ TRUE_VAL
                if (!memcmp(a, b, len))
                    result ^= MASK2; // result = TRUE_VAL
            }
        }
    }

    return result;
}
```

FIGURE 2 – Securised version of memcmp in C

The first example code implemented in rust is `memcmps`. `memcmps` is a secure version of `memcmp` that is available in C.

`memcmps` takes two input arrays that are initialized before the function is called.
`memcmps` returned two possible values :

— `TRUE_VAL`, corresponding to `0x1234`, when both array are equal.

— `FALSE_VAL`, corresponding to `0x5678`, when they are not equal.

In Figure 2 we show one of the three versions of `memcmps` that we implemented. This version used four function calls of `memcmp` as shown in line 7,9,11 and 13 and two masks `MASK` and `MASK2`, to prevent fault injection from affecting the returned value of the function.

— `MASK` corresponding to the value `0xABCD`

— `MASK2` corresponding to the value `0xF4F4`

`memcmps` works by using `MASK` on a variable called result (see line 5 on Figure 2).
If no fault is injected into the system, `memcmps` should return either true of false every time it is called.
So if both data blocks are the same, then result will take the value of `MASK` after the first call at line 8.
the value take `MASK` $\wedge$ `MASK2` after the second call line 10.
the value take `MASK2` $\wedge$ `TRUE_VAL` after the third call line 12.
And finally the value take `TRUE_VAL` after the fourth call line 14.

We studied two attack objective with the help of an oracle :

— returning `TRUE_VAL` when both data blocks are **different**.

— returning anything but `FALSE_VAL` when both data block are equal.

The attack model used is the inversion test and data mutation,
in the figure 2 one inversion test may cause the value to be neither `TRUE_VAL` nor `FALSE_VAL` (if the attack is done on the condition test line 7). Four inversion tests are needed to be able to return `TRUE_VAL` when the data blocks are different.

We implemented the Rust version of the code that would be written by C programmers.

Figure 3 shows the `memcmps` source code equivalent of Figure 2 in the Rust language.
In the Rust version of `memcmps`, we needed to change the return value of `memcmp`, due to C not having native boolean type.
`memcmp` returns *false* if both binary data blocks are the same and *true* otherwise.
We also used two flags for compiling Rust :

— The first is the use of `no_mangle` flag on line 17 in Figure 3.
It is used to prevent Rust from renaming some of the blocks generated by the LLVM and to ensure the correct operation of Lazart.

— The second is the use of `inline(never)` on line 18, which forces rust not to replace the function call with its body.

```
17  #[no_mangle]
18  #[inline(never)]
19  pub fn memcmps(a: &[u8; crate::common::SIZE] , b : &[u8;
    crate::common::SIZE], len: i32) -> i32{
20      let mut result = crate::common::FALSE_VAL;
21      let len_value = crate::do_real_load!(len);
22      let result_value = crate::do_real_load!(result);
23
24      if !memcmp(a, b, len as u64) {
25          result ^= crate::common::FALSE_VAL ^
            crate::common::MASK; // result = MASK
26          if !memcmp(a, b, len as u64) {
27              result ^= crate::common::MASK2; // result =
                MASK ^ MASK2
28              if !memcmp(a, b, len as u64) {
29                  result ^= crate::common::TRUE_VAL ^
                    crate::common::MASK; // result = MASK2 ^
                    TRUE_VAL
30                  if !memcmp(a, b, len as u64){
31                      result ^= crate::common::MASK2; //
                        result = TRUE_VAL
32                  }
33              }
34          }
35      }
36      result
37  }
38
```

FIGURE 3 – Securised version of memcmp in Rust

This is a type of optimisation used by compilers to avoid context changes.

5.4 verifypin

```
1   #include "verify_pin.h"
2   #include "lazart.h"
3
4   BOOL compare(uint8_t* a1, uint8_t* a2, size_t size)
5   {
6       int i;
7       __LZ__RENAME_BB("for_in_compare");
8       for (i = 0; i < size; i++) {
9           __LZ__RENAME_BB("check in compare");
10          if (a1[i] != a2[i]) {
11              return FALSE;
12          }
13      }
14      return TRUE;
15  }
16
17  BOOL verify_pin(uint8_t* user_pin)
18  {
19      __LZ__RENAME_BB("Try counter < 0");
20      if (try_counter > 0) {
21
22          __LZ__RENAME_BB("precompare");
23          if (compare(user_pin, card_pin, PIN_SIZE)) {
24              try_counter = TRY_COUNT;
25              return TRUE;
26          } else {
27              __LZ__RENAME_BB("precompare_else");
28              try_counter--;
29              return FALSE;
30          }
31      }
32      return FALSE;
33  }
```

FIGURE 4 – Verification of PIN code in C

The second example code implemented in Rust is **verifypin**.
The `verify_pin` program implements the PIN verification process of a smart card.
The source code in C shown in Figure 4 corresponds to a simple implementation without countermeasures other than a hardened boolean.

The **verifypin** function consists of two parts :

— Checking if there are enough tries remaining (line 20 in Figure 4).

— Resetting the try counter and returning `TRUE` if the `compare` function returns `TRUE`; otherwise, decrementing the number of tries remaining by 1 and returning `FALSE`.

`TRUE` contains the value `0xAA`.
`FALSE` contains the value `0x55`.

We consider `TRUE` and `FALSE` as hardened boolean values because they are encoded not just as single 0 or 1 bits, but as 8-bit patterns of 0s and 1s. Specifically, the pattern '01' represents `FALSE`, while '10' represents `TRUE`.
This approach makes it highly unlikely for a `TRUE` value to be accidentally flipped to `FALSE`, or vice versa.

The function takes one user PIN and returns either `TRUE` or `FALSE`.

The function **compare**, shown in Figure 4, compares each digit of the user input PIN with the one stored on the card. It returns `TRUE` if both PIN codes are identical and `FALSE` otherwise.

We studied two attack objectives with the help of an oracle :

— Returning `TRUE` when both PIN codes are different.

— Increasing the number of tries remaining in the case where both PIN codes are different.

The attack model used is the **test inversion**.

We implemented the Rust version of the source code shown in Figure 4, following the style used by C programmers (see Figure 5).

6 Idomatic rust

7 result

add devrait se remplir facilement en reprenant les résultats FDTC

8 Conclusion

```rust

#[no_mangle]
#[inline(never)]
#[allow(unused_mut)]
pub fn compare(a1: &[u8;crate::common::PIN_SIZE], a2: &[u8;
crate::common::PIN_SIZE], size: usize) ->
crate::common::sec_bool_t {
    let mut result = crate::common::sec_bool_t::TRUE;
    crate::lazart::_LZ__RENAME_BB_R("for_in_compare\0");
    for i in 0..size{
        crate::lazart::_LZ__RENAME_BB_R("check in compare\0");
        if a1.get(i) != a2.get(i) { // IP0 / IP1 / IP2 / IP3 ->
        loop unrolling from the rustc compiler.
            result = crate::common::sec_bool_t::FALSE;
        }
    }
    result
}

#[no_mangle]
#[inline(never)]
pub fn verify_pin(user_pin: &[u8; crate::common::PIN_SIZE],
card_pin: &[u8; crate::common::PIN_SIZE], try_counter: &mut u8)
-> crate::common::sec_bool_t {
    crate::lazart::_LZ__RENAME_BB_R("Try counter < 0\0");
    if  *try_counter > 0{ // IP4

        crate::lazart::_LZ__RENAME_BB_R("precompare\0");
        if compare(a1: user_pin, a2: card_pin,
        size: crate::common::PIN_SIZE).eq(&
        crate::common::sec_bool_t::TRUE){ // IP5
            // Authentication
            *try_counter = crate::common::TRY_COUNT as u8;
            return crate::common::sec_bool_t::TRUE;
        } else {
            crate::lazart::_LZ__RENAME_BB_R("precompare_else\0");
            *try_counter = *try_counter - 1 ;
            return crate::common::sec_bool_t::FALSE;
        }
    }

    crate::common::sec_bool_t::FALSE
}

```

FIGURE 5 – Verification of PIN code in Rust

9 Reference

[1] Z. Segall et al. « FIAT - Fault injection based automated testing environment ». In : *Twenty-Fifth International Symposium on Fault-Tolerant Computing, 1995, ' Highlights from Twenty-Five Years'.* 1995, p. 394-. doi : 10.1109/FTCSH.1995.532663.

[2] Johannes Obermaier, Robert Specht et Georg Sigl. « Fuzzy-glitch : A practical ring oscillator based clock glitch attack ». In : *2017 International Conference on Applied Electronics (AE).* IEEE. 2017, p. 1-6.

[3] Thomas Korak et Michael Hoefler. « On the effects of clock and power supply tampering on two microcontroller platforms ». In : *2014 Workshop on Fault Diagnosis and Tolerance in Cryptography.* IEEE. 2014, p. 8-17.

[4] Loic Zussa et al. « Investigation of timing constraints violation as a fault injection means ». In : *27th Conference on Design of Circuits and Integrated Systems (DCIS), Avignon, France.* Citeseer. 2012, p. 1-6.

[5] Loic Zussa et al. « Analysis of the fault injection mechanism related to negative and positive power supply glitches using an on-chip voltmeter ». In : *2014 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST).* IEEE. 2014, p. 130-135.

[6] Nidhal Selmane et al. « Security evaluation of application-specific integrated circuits and field programmable gate arrays against setup time violation attacks ». In : *IET information security* 5.4 (2011), p. 181-190.

[7] Bodo Selmke et al. « Precise laser fault injections into 90 nm and 45 nm sram-cells ». In : *Smart Card Research and Advanced Applications : 14th International Conference, CARDIS 2015, Bochum, Germany, November 4-6, 2015. Revised Selected Papers 14.* Springer. 2016, p. 193-205.

[8] Jean-Max Dutertre et al. « Laser fault injection at the CMOS 28 nm technology node : an analysis of the fault model ». In : *2018 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC).* IEEE. 2018, p. 1-6.

[9] Amine Dehbaoui et al. « Injection of transient faults using electromagnetic pulses Practical results on a cryptographic system ». In : *ACR Cryptology ePrint Archive (2012)* (2012).

[10] Amine Dehbaoui et al. « Electromagnetic transient faults injection on a hardware and a software implementations of AES ». In : *2012 Workshop on Fault Diagnosis and Tolerance in Cryptography.* IEEE. 2012, p. 7-15.

[11] Yoongu Kim et al. « Flipping bits in memory without accessing them : An experimental study of DRAM disturbance errors ». In : *ACM SIGARCH Computer Architecture News* 42.3 (2014), p. 361-372.

[12] Louis Dureuil et al. « FISSC : a fault injection and simulation secure collection ». In : *Computer Safety, Reliability and Security.* T. 9922. Lecture Notes in Computer Science. Trondheim, Norway : Springer, sept. 2016, p. 3-11. doi : 10.1007/978-3-319-45477-1_1. url : https://hal.science/hal-03784107.

[13] Christopher Simon Liu et al. *SoK : A Beginner-Friendly Introduction to Fault Injection Attacks.* 2025. arXiv : 2509.18341 [cs.CR]. url : https://arxiv.org/abs/2509.18341.

[14] Jan Richter-Brockmann et al. « FIVER – Robust Verification of Countermeasures against Fault Injections ». In : *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021.4 (août 2021). Artifact available at https://artifacts.iacr.org/tches/2021/a16, p. 447-473. doi : 10.46586/tches.v2021.i4.447-473.

[15] J. Arlat et al. « Fault injection for dependability validation : a methodology and some applications ». In : *IEEE Transactions on Software Engineering* 16.2 (1990), p. 166-182. doi : 10.1109/32.44380.

[16] Victor Arribas et al. « Cryptographic Fault Diagnosis using VerFI ». In : *2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST).* 2020, p. 229-240. doi : 10.1109/HOST45689.2020.9300264.

[17] William Pensec, Vianney Lapôtre et Guy Gogniat. *Automating Fault Injection through CABA Simulation for Vulnerability Assessment.* Colloque National du GDR SoC2. Poster. GDR SoC2, juin 2024. url : https://hal.science/hal-04727380.

[18] Marie-Laure Potet et al. *Lazart : A Symbolic Approach for Evaluation the Robustness of Secured Codes against Control Flow Injections [SDTA14 version].* Déc. 2015.

[19] Lionel Rivière et al. « Combining High-Level and Low-Level Approaches to Evaluate Software Implementations Robustness Against Multiple Fault Injection Attacks ». In : *Foundations and Practice of Security - 7th International Symposium, FPS 2014, Montreal, QC, Canada, November 3-5, 2014. Revised Selected Papers.* 2014, p. 92-111.

[20] Minjae KIM, Eunsung LEE et Jaeyong LEE. « Hardening Rust's Result : Simple Source Code Transformations for Fault-Robust Bootloader ». In : *2026 IEEE International Conference on Consumer Electronics (ICCE)*. 2026, p. 1-5. DOI : 10.1109/ICCE67443.2026.11449654.

[21] Jacopo SINI et al. « Improving Software Reliability with Rust : Implementation for Enhanced Control Flow Checking Methods ». In : *2025 Design, Automation & Test in Europe Conference (DATE)*. 2025, p. 1-7. DOI : 10.23919/DATE64628.2025.10992995.

[22] Sébastien MICHELLAND. « Compilation pour la sécurité matérielle : au-delà de la sémantique ». s343016. Thèse de doct. 2025. URL : /document.

[23] Cristian CADAR, Daniel DUNBAR et Dawson R. ENGLER. « KLEE : Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs ». In : *USENIX Symposium on Operating Systems Design and Implementation*. 2008. URL : https://api.semanticscholar.org/CorpusID:2520229.

[24] *KLEE — klee-se.org*. https://klee-se.org/. [Accessed 01-06-2026].